2026
Software Architecture Specifications
Uni Textbook Marketplace

For **Demo 3**

Presented by:

NexusDev

Software Architecture Specifications (SAS)

Project:	Uni Textbook Marketplace
Team:	NexusDev
Client:	Agile Bridge
University:	University of Pretoria - COS 301 Software Engineering 2026
Document version:	2.0 - Demo 3
Last updated:	4 September 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1 Introduction **5**

2 Architectural Requirements **6**
2.1 Architectural Patterns . . . 6
2.2 Design Patterns . . . 7
2.3 Constraints . . . 7
2.4 Architectural Diagram . . . 8
2.5 Mapping Quality Requirements to Architectural Decisions . . . 8

3 Technology Requirements **10**

4 API Contracts **12**
4.1 Carried Over from Demo 1 (confirmed still in use) . . . 12
4.2 Auth API Contracts . . . 12
4.2.1 POST /auth/register . . . 12
4.2.2 POST /auth/verify-otp . . . 12
4.2.3 POST /auth/login . . . 13
4.2.4 POST /auth/set-password . . . 13
4.2.5 GET /auth/me . . . 14
4.2.6 POST /auth/logout . . . 14
4.2.7 GET /auth/universities . . . 14
4.2.8 POST /auth/refresh . . . 14
4.3 Listings API Contracts . . . 14
4.3.1 POST /listings . . . 14
4.3.2 GET /listings . . . 15
4.3.3 GET /listings/mine . . . 15
4.3.4 GET /listings/:id . . . 15
4.4 Admin API Contracts . . . 15
4.4.1 GET /listings/admin/all . . . 15
4.4.2 PATCH /admin/listings/:id/approve . . . 16
4.4.3 PATCH /admin/listings/:id/reject . . . 16
4.5 Modules API Contract . . . 16
4.5.1 POST /modules . . . 16
4.5.2 GET /modules/faculties . . . 16
4.6 New for Demo 2 . . . 16
4.7 Listing API Contract . . . 16
4.7.1 PATCH /listings/editlist . . . 16
4.8 Module API Contract . . . 17

4.8.1 GET /modules (extended filters) . . . 17
4.9 Saved Searches API Contract . . . 17
4.9.1 POST /saved-searches/ . . . 17
4.9.2 GET /saved-searches/me . . . 18
4.9.3 GET /saved-searches/id . . . 18
4.9.4 DELETE /saved-searches/id . . . 18
4.9.5 GET /saved-searches/me/email . . . 18
4.9.6 GET /saved-searches/me/role . . . 19
4.10 Wishlist API Contract . . . 19
4.10.1 POST /wishlist/:listingId . . . 19
4.10.2 DELETE /wishlist/:listingId . . . 19
4.10.3 GET /wishlist/mine . . . 19
4.11 Admin API Contract . . . 19
4.11.1 GET /admin/audit-log . . . 20
4.11.2 GET /admin/emails . . . 20
4.12 Image API Contract . . . 20
4.12.1 POST /images/upload . . . 20
4.12.2 GET /images/list . . . 20
4.12.3 DELETE /images/delete . . . 20
4.13 Conversations API Contract . . . 21
4.13.1 POST /conversations . . . 21
4.13.2 POST /conversations/mine . . . 21
4.13.3 POST /conversations/:id/messages . . . 21
4.13.4 GET /conversations/:id/messages . . . 21
4.14 Demo 3 API Contracts: . . . 22
4.14.1 POST /reports . . . 22
4.14.2 GET /admin/reports . . . 22
4.15 Admin Moderation API Contract . . . 22
4.15.1 PATCH /admin/reports//dismiss . . . 22
4.15.2 PATCH /admin//ban . . . 22
4.16 Notifications API Contract . . . 23
4.16.1 GET /notifications/mine . . . 23
4.16.2 PATCH /notifications/:id/read . . . 23
4.16.3 PATCH /notifications/read-all . . . 23
4.16.4 DELETE /notifications/:id/delete . . . 23

5 Non-Functional Requirements (NFR) Testing 24
5.1 Testing Overview . . . 24
5.2 NFR Traceability Matrix . . . 24

5.3 Test Results Summary . . . 26
5.4 Load Test Results . . . 26
5.5 Accessibility Test Results . . . 27
5.6 Testing Tools Used . . . 27
5.7 Known Issues and Action Plan . . . 28
5.7.1 Critical Issues (Need Immediate Fix) . . . 28
5.7.2 Untested NFRs (Need Testing) . . . 28

6 Deployment **28**
6.1 Overview . . . 28
6.2 Live URLs . . . 29
6.3 Deployment Diagram . . . 30
6.4 CI/CD Pipeline Diagram . . . 30
6.5 Deployment Requirements Status . . . 32
6.6 Rollback Strategy . . . 32
6.7 Issues Encountered and the Decisions That Came Out of Them . . . 32

1. Introduction

The Uni Textbook Marketplace is a web app that lets verified university students buy, sell, and swap second-hand textbooks in a way that's actually organised around modules, instead of just being a generic classifieds board. The SRS explains what the system needs to do. This document is about how we actually built it to do that: the architectural and design patterns we went with, the tech stack, our API contracts, and how everything gets deployed.

Roughly speaking, the backend is a NestJS modular monolith, the frontend is Next.js, and Postgres (hosted through Azure) handles the data. Messaging is kept separate and runs through Firebase Firestore instead. Everything runs on Azure Container Apps, split into a production and a staging environment, and GitHub Actions handles both the quality checks and the actual deployment.

2. Architectural Requirements

2.1 Architectural Patterns

Modular Monolith (Core Backend)

The backend is a NestJS modular monolith. Rather than splitting everything into separate microservices right from the start, all the core domain logic (Auth, Listings, Moderation, Modules, Saved Search, Wishlist, Audit Log) lives in one NestJS app, just split into bounded modules. Each module has its own controller, service, and module file, and modules aren't allowed to inject each other's services directly. We went with this mainly because we're a small team and the domain is pretty well understood at this stage, so going full microservices this early would just be extra operational overhead for not much gain. Since the modules are already cleanly separated, if we ever did need to split into microservices later, it wouldn't mean a big rewrite.

External Microservice (Messaging)

In-app messaging (UC4) runs through Firebase Firestore as its own separate microservice, kept outside the core monolith on purpose. The frontend talks to Firestore directly through the client SDK, so the NestJS backend isn't sitting in the middle of that traffic at all. That means if Firestore ever goes down, it doesn't take the rest of the platform with it. That's a graceful degradation boundary we built in deliberately, not something that just happened.

Stateless API Layer

The NestJS API doesn't keep any session state on the server side. Auth works through a JWT plus an HTTP-only cookie, and the cookie settings change depending on the environment (`sameSite: 'none'` and `secure: true` once it's deployed, since frontend and backend sit on different hostnames there, and `sameSite: 'lax'` locally where that isn't an issue). Because there's no server-side session state, any backend instance can handle any request, which is what actually makes horizontal scaling possible without needing sticky sessions.

Read-Heavy Performance Strategy

People browse and search listings way more than they create new ones, so the schema has indexes on the columns that get used most in filters and sorting: a composite index on `(module_id, price)`, plus separate indexes on `condition`, `annotation_level`, `status`, `seller_id`, `reviewed_by`, and `created_at`. Books get indexed on `isbn`, `(author, title)`, and `edition`. Audit log entries are indexed on `(entity_type, entity_id)` and `performed_at`.

Repository Pattern (Data Access)

Services never write raw SQL directly. All database access goes through TypeORM repositories, so the actual Postgres queries stay abstracted away from the business logic. This is basically the object-persistence pattern we covered in the architecture lectures (L11), where a DB manager layer sits between the business objects and the actual database, so the business logic doesn't need to know or care what's underneath it.

2.2 Design Patterns

- **Decorator Pattern.** NestJS guards (`@UseGuards(JwtAuthGuard, RolesGuard)`) add auth checks onto a controller method without touching what the method actually does. Keeps the security stuff separate from the endpoint's actual job.
- **Builder Pattern.** TypeORM's `createQueryBuilder()` is used for listing search. We start with a base query and keep adding `andWhere()` clauses depending on which filters were actually sent in the request, instead of writing a separate hardcoded query for every possible combination of filters.
- **Factory Pattern.** `ListingResponseFactory` is a small factory that turns `Listing` entities into the objects the API actually returns, adding stuff like a formatted price or a status label that isn't stored in the DB itself. Keeps the response shape the same across every listing endpoint.

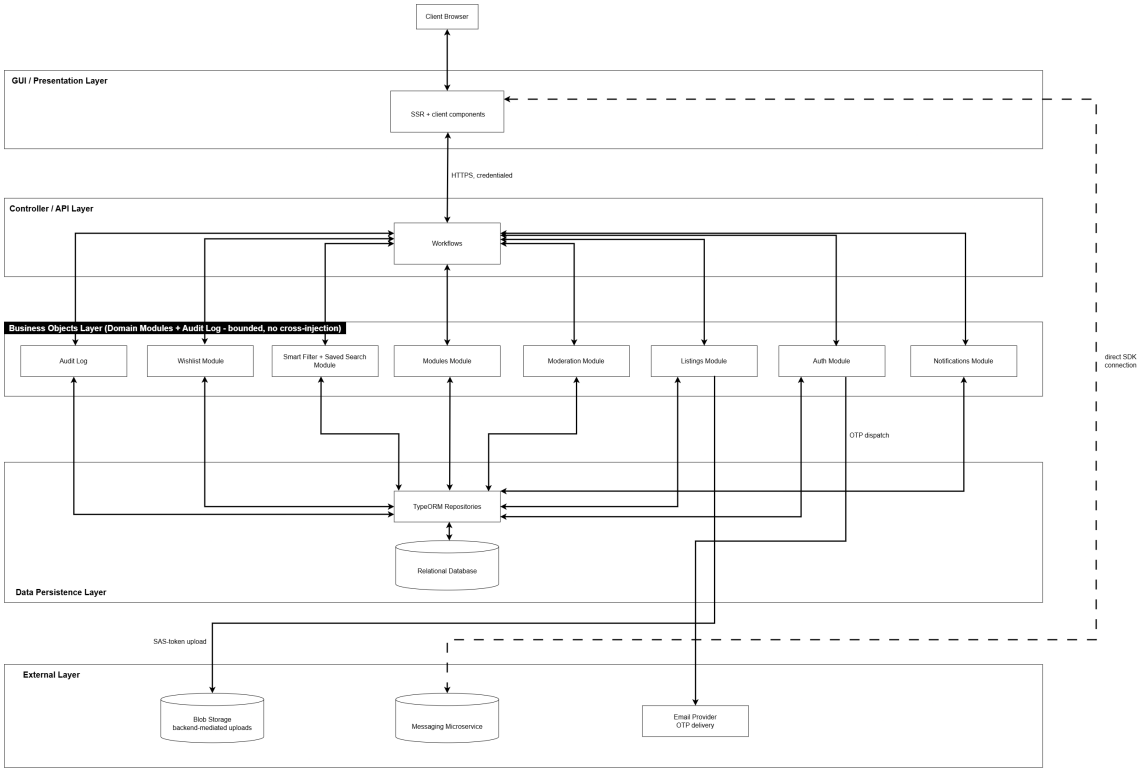
2.3 Constraints

Constraint	Detail
Hosting	Azure, under Agile Bridge's subscription (through our client contact Kyle). We manage the actual deployment but not the subscription/tenant itself.
Authentication	Custom OTP email verification, not a managed identity provider like Azure B2C.
Email allowlist	`@tuks.co.za` only, for now. It's configurable through backend environment variables.
No payment processing	The platform only sets up meetup-based exchanges. No payment gateway, escrow, or any kind of financial transaction.
ISBN metadata	Entered manually, no external ISBN lookup API.
No mobile app	Web only for now. A React Native app is more of a stretch goal, not something we're actually building.

Constraint	Detail
Multi-university support	Only `tuks.co.za` is actually set up right now, even though the data model itself could support more than one university.

2.4 Architectural Diagram

This is a tech-neutral view of the main components in the system and how they talk to each other.



Notice how messaging doesn't go through the NestJS gateway at all, that's the graceful degradation boundary from §2.1, shown visually here.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
Fast multi-filter listing search	Composite and single-column Postgres indexes on the columns used for listing search and sorting. Building the query with the builder pattern avoids unnecessary full-table scans.

Quality Requirement	**Architectural Decision**
Core features stay up if messaging fails	Messaging is kept separate as an external Firestore microservice, hit directly by the frontend, so if Firestore goes down it can't cascade into Auth, Listings, or Moderation.
Horizontal scalability	Stateless JWT-based auth means any backend instance can serve any request. Azure Container Apps can just add replicas without needing sticky sessions.
Only verified students can create listings, only admins can moderate	OTP verification happens at registration. `JwtAuthGuard` and `RolesGuard` (Decorator pattern) sit on every protected route.
Modular, low-coupling codebase	NestJS bounded modules (Auth, Listings, Moderation, Modules, Saved Search, Wishlist, Notifications, Audit Log), with no module directly injecting another module's service.
WCAG AA accessibility	Checked through automated Lighthouse audits. Currently around 95% AA compliant.
Measurable backend test coverage	`ci.yml` runs `npm run test:cov` against a real, temporary Postgres 15 container on every push and PR to `main`/`develop`. Coverage gets uploaded to Codecov.
Code quality and static analysis	SonarCloud runs on every push to `main`/`develop` and on every PR targeting either branch.
Reproducible, environment-parity deployment	Two fully separate Container App pairs for production and staging, each with its own database, both deployed automatically through SHA-tagged images (see §5).

3. Technology Requirements

Layer	Technology	Justification
Frontend	Next.js, React, TypeScript	Server-side rendering helps with how fast the listing browse pages feel, which are the pages that get the most traffic. TypeScript keeps the codebase type-safe throughout.
Frontend styling	Tailwind CSS	Utility-first styling means we can iterate faster, and the token-based theming keeps things consistent with the Agile Bridge brand style guide.
Backend	NestJS, TypeScript	The module structure lines up directly with our bounded-module architecture. Decorators cut down on repetitive guard and DTO code.
Database	Azure Database for PostgreSQL (Flexible Server)	Relational data suits our domain well, and it's fully ACID-compliant. Being Azure-native also fits our hosting constraint.
ORM	TypeORM	Gives us the repository pattern for data access, plus a migration system for changing the schema over time.
Authentication	Custom OTP plus JWT	Matches the client's OTP verification requirement. JWT is stateless, so we're not tied to a managed identity provider.
File storage	Azure Blob Storage, backend-mediated with SAS tokens	Uploads go through the backend first instead of straight to Blob Storage, so storage credentials never touch the client.
Messaging	Firebase Firestore	Handles real-time chat as its own external microservice. If it goes down, the rest of the platform keeps working. The SDK also handles real-time subscriptions on its own, no backend proxying needed.

Layer	Technology	Justification
Hosting	Azure Container Apps	Container-native, and it has a built-in revision/rollback model that we've actually had to use, which is a big part of why we chose it over App Service.
Container Registry	Azure Container Registry	Private image storage. `az acr build` lets us build remotely without needing Docker running locally.
CI (lint, test, coverage)	GitHub Actions, `ci.yml`	Runs on every push and PR to `main`/`develop`. Backend tests run against a real, temporary Postgres container instead of a mock, so we trust the results more.
CD (build, deploy)	GitHub Actions, `deploy.yml`	Branch-conditional, `main` deploys to production and `develop` deploys to staging. Images are tagged with the commit SHA instead of `:latest`, so Container Apps always picks up a real change.
Code quality and static analysis	SonarCloud	Runs on every push to `main`/`develop` and on every PR targeting either branch.
Coverage reporting	Codecov	Separate `backend` and `frontend` flags per CI job.
Testing (backend)	Jest, plus a real temporary Postgres container in CI	Gives us integration-level confidence instead of just relying on a mocked database.
Testing (frontend)	Jest unit tests	Cypress end-to-end tests are planned but we haven't gotten to them yet.
API documentation	Swagger (OpenAPI), generated from NestJS decorators	Auto-generated, interactive API docs straight from the controller code.

4. API Contracts

All endpoints are prefixed with `/api`. Every authenticated endpoint needs the header `Authorization: Bearer <JWT>`. All request and response bodies use `Content-Type: application/json`.

4.1 Carried Over from Demo 1 (confirmed still in use)

4.2 Auth API Contracts

4.2.1 POST /auth/register

Description: Starts student registration. Checks the email domain and sends out the OTP.
Auth required: No

Request body:

```
{
  "fullName": "string (required)",
  "surname": "string (required)",
  "university": "string (required)",
  "email": "string (required, must be allowed domain)"
}
```

Success response (201 Created):

```
{
  "message": "Registration successful. Check your university email for
     verification code",
  "userId": "uuid"
}
```

Error responses:

- 400: Missing required field, or email domain not in allowlist
- 409: Email address is already registered

4.2.2 POST /auth/verify-otp

Description: Checks the 6-digit OTP sent to the student's email.
Auth required: No

Request body:

```
{
  "email": "string (required)",
  "otp": "string (required, 6 digits)"
}
```

Success response (200 OK):

```
{
  "message": "Email verified successfully.",
  "verified": true
}
```

Error responses:

- 400: OTP is incorrect or has expired
- 404: No pending OTP found for this email

4.2.3 POST /auth/login

Description: Logs in a verified student and hands back a signed JWT.
Auth required: No

Request body:

```
{
  "email": "string (required)",
  "password": "string (required, min 8 characters)"
}
```

Success response (200 OK):

```
{
  "message" : "Login sucessful."
}
```

Error responses:

- 401: Invalid email or password
- 403: Account exists but email is not verified

4.2.4 POST /auth/set-password

Description: Sets the student's password once OTP verification is done.
Auth required: No (uses `userId` from the registration step)

Request body:

```
{
  "userId": "uuid (required)",
  "password": "string (required, min 8 characters)",
  "confirmPassword": "string (required, must match password)"
}
```

Success response (200 OK):

```
{
  "message": "Password set successfully.",
  "accessToken": "string (JWT)"
}
```

4.2.5 GET /auth/me

Description: Returns the logged-in user's profile.
Auth required: Yes

4.2.6 POST /auth/logout

Description: Clears the session cookie.
Auth required: Yes

4.2.7 GET /auth/universities

Description: Lists supported universities, for the registration dropdown.
Auth required: No

4.2.8 POST /auth/refresh

Description: Gets a new access token using a refresh token, so users stay logged in without having to type their password again.
Auth required: No

4.3 Listings API Contracts

4.3.1 POST /listings

Description: Creates a new textbook listing. Gets created with `status = PENDING`.
Auth required: Yes (Student role)

Request body:

```
{
  "isbn": "string (optional)",
  "title": "string (required)",
  "author": "string (optional)",
  "edition": "integer (required)",
  "condition": "new | good | fair | poor (required)",
  "annotationLevel": "none | light | heavy (required)",
  "bookId": "uuid (required)",
  "moduleId": "uuid (required)",
  "price": "number, positive decimal (required)",
  "hasNotes": "boolean (required)",
  "photoUrls": ["string (required >= 4)"],
  "description: "string (required)"
}
```

4.3.2 GET /listings

Description: Returns all APPROVED listings, with pagination and filters.
Auth required: Yes
Query parameters: `moduleId`, `faculty`, `condition`, `minPrice`, `maxPrice`, `edition`, `page`, `limit`

4.3.3 GET /listings/mine

Description: Returns all of the logged-in student's own listings, including PENDING, APPROVED and REJECTED ones.
Auth required: Yes (Student role)

4.3.4 GET /listings/:id

Description: Returns the full detail object for one listing.
Auth required: Yes
Path parameter: `id`, the UUID of the listing

4.4 Admin API Contracts

4.4.1 GET /listings/admin/all

Description: Returns all listings, admin only.
Auth required: Yes (Admin role only)

4.4.2 PATCH /admin/listings/:id/approve

Description: Approves a pending listing. Sets status to APPROVED and writes an audit log entry.
Auth required: Yes (Admin role only)

4.4.3 PATCH /admin/listings/:id/reject

Description: Rejects a pending listing. Sets status to REJECTED and writes an audit log entry, a reason is included with the rejection.
Auth required: Yes (Admin role only)

Note: The audit log write was added for Demo 2

4.5 Modules API Contract

4.5.1 POST /modules

Description: Creates a module linked to the textbook being sold. Takes module details like name, code, semester, and faculty id.

4.5.2 GET /modules/faculties

Description: Returns faculty names and ids. Used when creating a module during listing creation

4.6 New for Demo 2

4.7 Listing API Contract

4.7.1 PATCH /listings/editlist

Module: Listings (UC5)
Description: Edits the details of an existing listing that the student owns. Only works on listings that are rejected or pending.
Auth required: Yes (Student role, listing owner)
Request body: Only listing details can be edited, everything is optional except the listing id.

```
{
 "id" : "string (uuid)",
 "title" : "string",
 "condition" : 'new' | 'good' | 'fair' | 'poor',
```

```
 "annotation_level" : 'none' | 'light' | 'heavy',
 "price" : "number",
 "has_notes" : "boolean",
 "description" : "string",
}
```

4.8 Module API Contract

4.8.1 GET /modules (extended filters)

Module: Smart Filters (UC6)
Description: Extends the existing module search endpoint with extra smart filter parameters.
Description: Search-as-you-type module code lookup. Returns modules matching the search query.
Auth required: Yes
Query parameters:

- `search` (string, required): partial module code or name to search
- `university` (string, optional): filter by university name (e.g. `"UP"`)

4.9 Saved Searches API Contract

4.9.1 POST /saved-searches/

Module: Saved Search (UC6-ext)
Description: Saves a search as JSON **Request body:** all fields are optional

```
{
 "filter_json" : {
 "faculty" : "string",
 "moduleCode" : "string",
 "edition" : "string",
 "priceMin : "string | number",
 "priceMax" : "string | number",
 "condition" : "string",
 "annotationLevel" : "string",
 "search" : "string",
 "book_title" : "string",
 "author" : "string",
 "isbn" : "string",
 "university_id" : "string",
```

```
 "faculty_id": "string"
 }
}
```

4.9.2 GET /saved-searches/me

Module: Saved Search (UC6-ext)
Description: Returns the saved searches belonging to the logged-in student. Confirmed to exist as a route in `saved_search.controller.ts`.
Auth required: Yes
Request body: uses query parameters.

- `id` (string)
- `email` (string)
- `role` (string)
- `firstname` (string)
- `lastname` (string)

4.9.3 GET /saved-searches/id

Module: Saved Search (UC6-ext)
Description: Returns one saved search by id **Auth required:** Yes
Request body: uses query parameters.

- `id` (string)

4.9.4 DELETE /saved-searches/id

Module: Saved Search (UC6-ext)
Description: Deletes a saved search by id **Auth required:** Yes
Request body: uses query parameters.

- `id` (string)

4.9.5 GET /saved-searches/me/email

Module: Saved Search
Description: Returns the logged-in student's saved search email.

4.9.6 GET /saved-searches/me/role

Module: Saved Search
Description: Returns role-specific saved search settings for the logged-in user.

4.10 Wishlist API Contract

4.10.1 POST /wishlist/:listingId

Module: Wishlist (UC7)
Description: Adds a listing to the logged-in student's wishlist.
Auth required: Yes (Student role)
Request body:

```
{
 "userid" : "string (uuid)",
 "listingid" : "string(uuid)",
}
```

4.10.2 DELETE /wishlist/:listingId

Module: Wishlist (UC7)
Description: Removes a listing from the logged-in student's wishlist.
Auth required: Yes (Student role)
Request body:

```
{
 "userid" : "string (uuid)",
 "listingid" : "string(uuid)",
}
```

4.10.3 GET /wishlist/mine

Module: Wishlist (UC7)
Description: Returns the logged-in student's wishlisted listings.
Auth required: Yes (Student role)

4.11 Admin API Contract

4.11.1 GET /admin/audit-log

Module: Audit Log (UC8)
Description: Returns audit log entries for admin review.
Auth required: Yes (Admin role only)
Query parameters: all are optional

- `performedBy` (string): filter by which admin did the action
- `action` (string): approved a listing or rejected a listing
- `entityType` (string): what type the action was performed on, currently only listing
- `entityId` (string)
- `startDate` (string)
- `endDate` (string)

4.11.2 GET /admin/emails

Module: Audit Log (UC8)
Description: Returns a list of admin emails, used for filters **Auth required:** Yes (Admin role only)

4.12 Image API Contract

4.12.1 POST /images/upload

Module: Blob upload
Description: Uploads a listing image to Azure Blob Storage through the backend.
Auth required: Yes (Student role)

4.12.2 GET /images/list

Module: Blob upload
Description: Lists uploaded images for a listing.
Auth required: Yes

4.12.3 DELETE /images/delete

Module: Blob upload
Description: Deletes an uploaded listing image from Blob Storage.

Auth required: Yes (Student role, listing owner)

4.13 Conversations API Contract

4.13.1 POST /conversations

Module: Messaging
Description: Creates or fetches a conversation for a listing
Auth required: Yes
Request body:

```
{
  "listingId": "uuid"
}
```

4.13.2 POST /conversations/mine

Module: Messaging
Description: Gets all conversations for the logged-in user
Auth required: Yes

4.13.3 POST /conversations/:id/messages

Module: Messaging
Description: Gets all messages in a conversation
Auth required: Yes
Api param:

```
{
  name: 'id',
  description: 'Conversation ID',
}
```

4.13.4 GET /conversations/:id/messages

Module: Messaging
Description: Sends a message in a conversation
Auth required: Yes
Api param:

```
{
```

```
  name: 'id',
  description: 'Conversation ID',
}
```

Request body:

```
{
  "text" : "string"
}
```

4.14 Demo 3 API Contracts:

4.14.1 POST /reports

Module: Reports **Description:** Creates a new report **Auth required:** Yes **Request body:**

```
{
"listing_id": "uuid",
"reason": "Fraud | Misleading | Duplicate | Other"
}
```

4.14.2 GET /admin/reports

Module: Reports / Admin **Description:** Getter for reports. **Auth required:** Admin only **Query parameters:** Optional report filters.

Module: Reports / Admin **Description:** Specific getter for reports review. **Auth required:** Admin only

4.15 Admin Moderation API Contract

4.15.1 PATCH /admin/reports//dismiss

Module: Admin / Reports **Description:** Dismisses a report **Auth required:** Admin only

4.15.2 PATCH /admin//ban

Module: Admin / Moderation **Description:** Perform the ban on a report **Auth required:** Admin only **Request body:**

```
{
"reason": "string"
```

```
}
```

4.16 Notifications API Contract

4.16.1 GET /notifications/mine

Module: Notifications
Description: Returns the logged-in user's notifications, paginated.
Auth required: Yes
Query parameters:

- `page` (number, optional, default 1)
- `limit` (number, optional, default 10)

4.16.2 PATCH /notifications/:id/read

Module: Notifications
Description: Marks a single notification as read.
Auth required: Yes (owner only)
Path parameters:

- `id` (string, uuid): notification id

4.16.3 PATCH /notifications/read-all

Module: Notifications
Description: Marks all of the logged-in user's notifications as read.
Auth required: Yes

4.16.4 DELETE /notifications/:id/delete

Module: Notifications
Description: Deletes a notification belonging to the logged-in user.
Auth required: Yes (owner only)
Path parameters:

- `id` (string, uuid): notification id

5. Non-Functional Requirements (NFR) Testing

5.1 Testing Overview

Every quality requirement in the SRS has been quantified and verified with at least one executable, repeatable test. The table below maps each NFR to its architectural tactic and the test that validates it.

5.2 NFR Traceability Matrix

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-01	GET /listings search and browse return within 2s for 95% of requests under 50 concurrent users	Composite indexing, Query optimization, Pagination, Redis caching	k6	p(95) < 2000ms	FAILED
NFR-02	Notification delivery via bell icon reflects triggering event within 15s polling interval	Polling mechanism optimization, Real-time updates	k6	< 15s	UNTESTED
NFR-03	Stateless JWT-based API layer supports horizontal scaling without code changes	No session state on server, JWT tokens	Architecture Review	Pass	PASS
NFR-04	Production runs with min-replicas: 1, max-replicas: 2 on Azure Container Apps	Container orchestration, Auto-scaling	Azure CLI	Configured	PASS
NFR-05	Staging scales to zero when idle (min-replicas: 0) with cold-start delay 20s	Zero-scaling configuration	Azure CLI	< 30s	PASS
NFR-06	All protected endpoints enforce JWT-based authentication	JWT middleware, Token validation	k6	401/403 for invalid/no token	UNTESTED

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-07	Admin-only endpoints return 403 Forbidden for Student-role JWT	Role-based access control (RBAC)	k6	403 Forbidden	UNTESTED
NFR-08	Passwords hashed with bcrypt at cost factor 12	bcrypt encryption, Salt generation	Jest	Cost factor 12	PASS
NFR-09	Login, registration, password-change endpoints rate-limited to 5 req/min per client	Rate limiting middleware	k6	429 after 5 requests	UNTESTED
NFR-10	Access tokens expire after 15 minutes, refresh tokens after 7 days	JWT expiry configuration	Jest	15min / 7 days	PASS
NFR-11	Banned user (is_banned=true) rejected with 403 on every login attempt	User status check middleware	k6	403 Forbidden	PASS
NFR-12	Core features remain available if external messaging microservice is unavailable	Circuit breaker, Fallback mechanism	Jest + Fault Injection	No 5xx errors	UNTESTED
NFR-13	Production maintains at least 1 warm replica at all times	Minimum replica configuration	Azure CLI	1 replica	PASS
NFR-14	Failed deployment rollback-able to known-good revision in under 5 minutes	Azure revision traffic shifting	Azure CLI	< 5 minutes	UNTESTED
NFR-15	Backend test coverage measured on every push/PR via CI pipeline	CI/CD pipeline, Codecov integration	Codecov	80% coverage	FAILED

ID	Requirement	Tactic in SAS	Test / Tool	Target	Status
NFR-16	Every new endpoint has corresponding entry in openapi.yaml same day	API documentation, OpenAPI spec	Manual Review	100% documented	PASS
NFR-17	UI meets Google Lighthouse Accessibility score 85/100 on every audited page	Semantic markup, ARIA labels	axe-core / Lighthouse	85/100	PASS
NFR-18	New pages from Demo 3 onward score 90/100 on Lighthouse	Accessibility-first design	axe-core / Lighthouse	90/100	UNTESTED
NFR-19	Every moderation, ban, appeal-decision action writes exactly one immutable audit_log entry	Audit log middleware	Jest	1 row per action	PASS
NFR-20	Audit log entries record action, admin UUID, and timestamp	Structured logging	Jest	All fields present	PASS

5.3 Test Results Summary

Category	Total	PASS	FAIL	UNTESTED
Performance	2	0	1	1
Scalability	3	3	0	0
Security	6	3	0	3
Reliability/Availability	2	1	0	1
Maintainability	3	1	1	1
Accessibility	2	1	0	1
Auditability	2	2	0	0
TOTAL	**20**	**11**	**2**	**7**

5.4 Load Test Results

Test Configuration:

- **Duration:** 40.7 seconds
- **Virtual Users:** 5 concurrent (ramping from 1 to 5)
- **Total Iterations:** 25
- **Success Rate:** 80% (120 out of 150 checks passed)

Endpoint Performance:

Endpoint	Avg	p(95)	Status
/listings	3.62s	19.99s	FAILED
/auth (JWT)	60.47ms	265.51ms	PASS
/auth/universities	43.95ms	199.89ms	PASS

5.5 Accessibility Test Results

Pages Tested:

Page	Lighthouse Score	axe-core Violations
Landing Page	94/100	0
Create Listing	94/100	0
In-App Messaging	95/100	0
Wishlist	96/100	0
Listings Browse	89/100	0
Audit Log	85/100	0

5.6 Testing Tools Used

Tool	Purpose	NFRs Covered
k6	Load testing for performance metrics under concurrent users	QR-01, QR-02, QR-06, QR-07, QR-09, QR-11
Jest	Unit & integration tests for security, auditability, fault injection	QR-08, QR-10, QR-12, QR-19, QR-20
axe-core / Lighthouse	Automated accessibility scanning for WCAG 2.1 AA compliance	QR-17, QR-18
Codecov	Test coverage tracking on CI pipeline	QR-15

Tool	Purpose	NFRs Covered
Azure CLI	Infrastructure validation for scaling & availability	QR-03, QR-04, QR-05, QR-13, QR-14
Manual Review	OpenAPI specification verification	QR-16

5.7 Known Issues and Action Plan

5.7.1 Critical Issues (Need Immediate Fix)

- **NFR-01:** Listings endpoint p95 latency at 19.99s (target <2000ms)
  - **Root Cause:** Missing database indexes on listings table, potential N+1 query pattern
  - **Action:** Add composite indexes, implement eager loading, add Redis caching
- **NFR-15:** Test coverage unknown (target 80%)
  - **Root Cause:** Coverage report not generated/pushed to Codecov
  - **Action:** Run `npm run test:cov` and push to Codecov

5.7.2 Untested NFRs (Need Testing)

- **NFR-02:** Notification polling 15s
- **NFR-06:** JWT authentication testing
- **NFR-07:** Admin RBAC testing
- **NFR-09:** Rate limiting testing
- **NFR-12:** Circuit breaker testing
- **NFR-14:** Rollback testing
- **NFR-18:** New page accessibility

6. Deployment

6.1 Overview

The system is deployed as two separate services, a NestJS backend and a Next.js frontend, both packaged into Docker images and run on Azure Container Apps. A managed Azure Database for PostgreSQL Flexible Server handles the data, and Azure Blob Storage handles listing image uploads through the backend, using SAS-token authentication.

There are two environments, production (served off `main`) and staging (served off `develop`), which is what covers the environment parity requirement. Both share one Container Apps Environment to keep costs down, but they're fully separated at the application and data level.

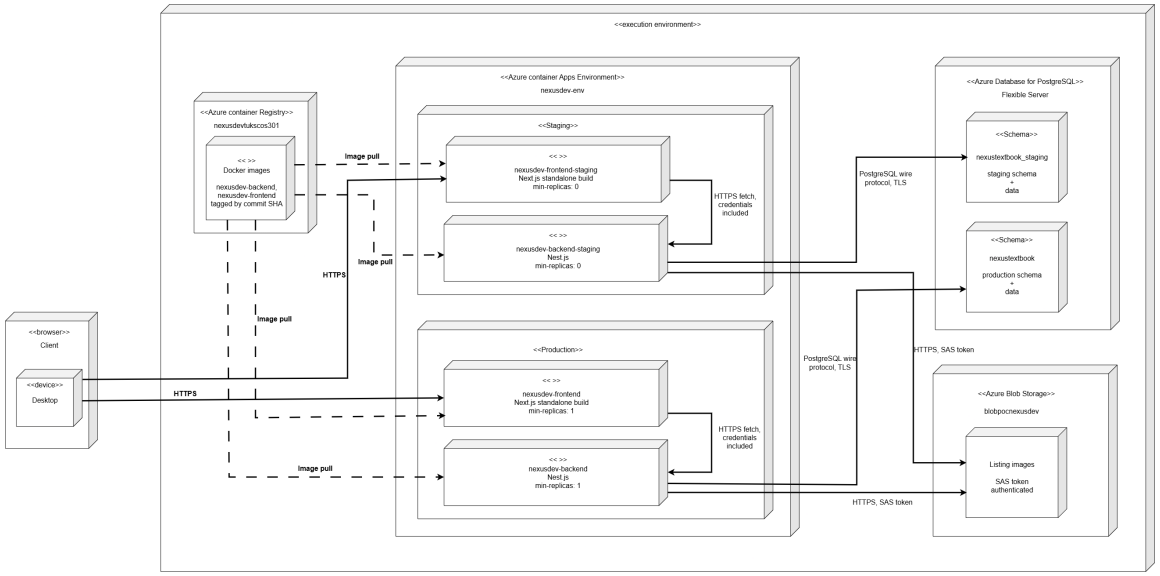
	Production	**Staging**
Trigger branch	`main`	`develop`
Backend app	`nexusdev-backend`	`nexusdev-backend-staging`
Frontend app	`nexusdev-frontend`	`nexusdev-frontend-staging`
Database	`nexustextbook`	`nexustextbook_staging`
Scaling	`min-replicas: 1`, always warm	`min-replicas: 0`, scales to zero when idle

We went with one shared Container Apps Environment and separate databases on purpose, instead of two fully separate environments. A second full environment felt like unnecessary cost for a student capstone project, and the part that actually matters for correctness, keeping the data isolated, is already handled by giving each environment its own Postgres database on the same server. Staging runs at `min-replicas: 0`, which means the first request after it's been idle a while takes a bit longer (a cold start), but it costs basically nothing while no one's using it. Production stays warm at `min-replicas: 1` since it needs to respond right away for real users at any time.

6.2 Live URLs

- **Production frontend:** https://nexusdev-frontend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io
- **Production backend:** https://nexusdev-backend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api
- **Staging frontend:** https://nexusdev-frontend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io
- **Staging backend:** https://nexusdev-backend-staging.whitesand-df72b78b.southafricanorazurecontainerapps.io/api

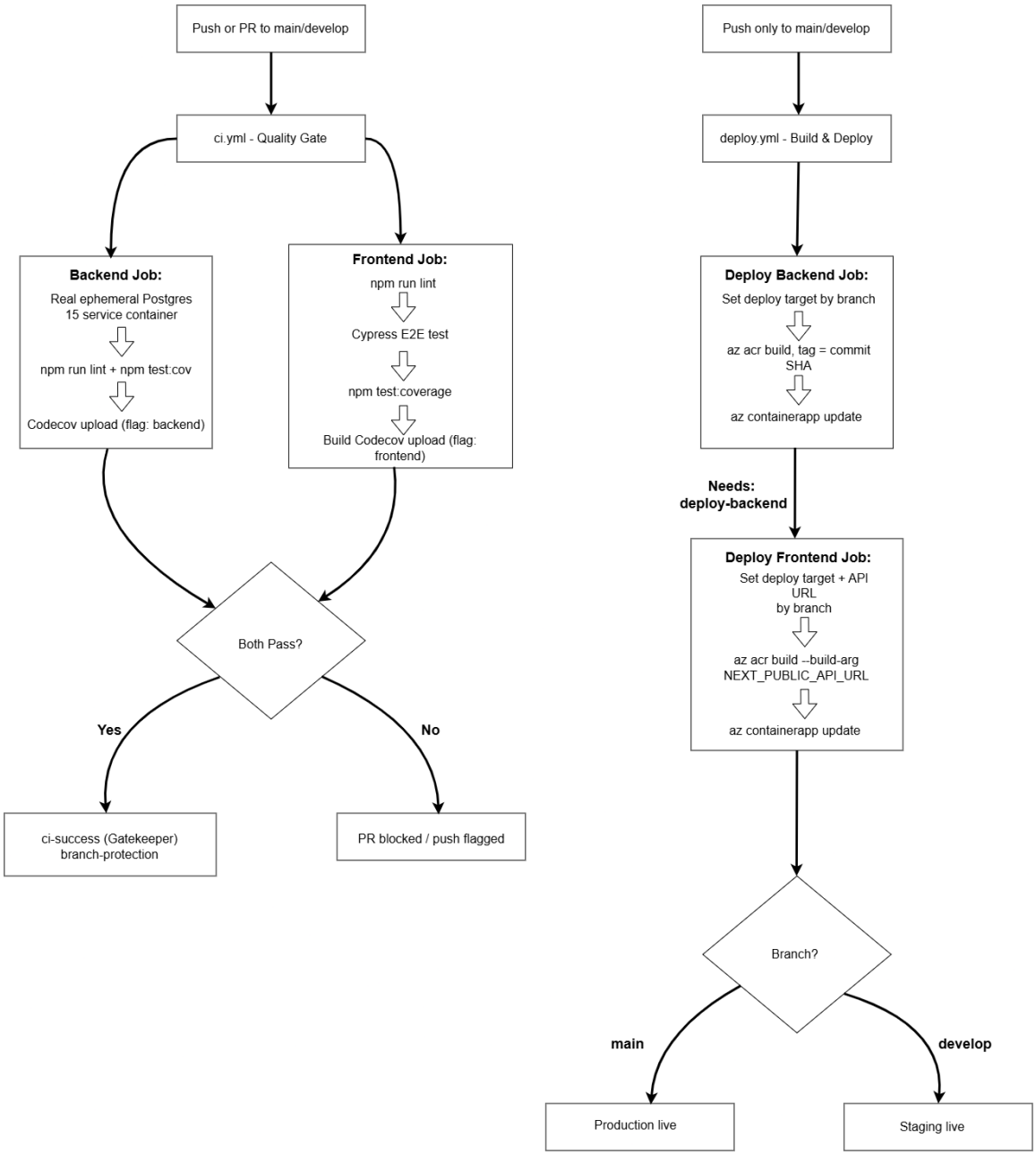
6.3 Deployment Diagram



This one diagram covers both environments, since they run on the exact same topology and only differ in which specific app and database instance they point at (labelled per node above). Region is South Africa North, resource group is `NexusDev`.

6.4 CI/CD Pipeline Diagram

There are two separate GitHub Actions workflows here, each doing a different job.



A few notes on how this actually works:

- `ci.yml` runs on both push and pull request to `main`/`develop`, this is the real quality gate. `deploy.yml` only runs on push, since an open PR shouldn't trigger an actual deployment.
- Branch protection is set up on both `main` and `develop`. Every merge has to come through a PR from a feature branch, needs at least one reviewer to approve it, and the `ci-success` check has to pass. A failed CI run blocks the merge outright.
- Inside `deploy.yml`, `deploy-frontend` only runs after `deploy-backend` succeeds, since the frontend build needs the backend's live URL as a build-time argument.
- Every image is tagged with the full commit SHA, never `:latest`. This was actually

a real bug we ran into early on, Container Apps sometimes wouldn't notice that a `:latest` image had changed, so we switched to SHA tags to make every deploy unambiguous.

6.5 Deployment Requirements Status

Requirement	Status	Notes
Live, accessible system	Done	Both environments reachable over public HTTPS (§5.2)
Environment parity	Done	Production and staging each have their own Container Apps, their own database, and their own CI/CD trigger
Infrastructure as Code / Containerization	Done	Docker multi-stage builds, deployed through Azure Container Apps
Secrets management	Done	All credentials live as Container Apps secrets or GitHub Actions secrets, never committed to the repo
Rollback strategy	Done	See §5.6

6.6 Rollback Strategy

Azure Container Apps keeps old revisions around. If a deployment goes wrong, we can just shift traffic back to the last known good revision without rebuilding anything:

```
az containerapp revision list --name nexusdev-backend --resource-group NexusDev
    --output table
az containerapp ingress traffic set --name nexusdev-backend --resource-group
   NexusDev --revision-weight <previous-revision-name>=100
```

This isn't just theoretical, we actually used this exact process during initial production deployment to fix a stale image that was causing CORS issues.

6.7 Issues Encountered and the Decisions That Came Out of Them

Issue	Root Cause	Decision Made
Stale `:latest` tag not picked up as changed by Container Apps	The tag string itself never changes, so the platform assumed nothing had	Every image now gets tagged with the commit SHA instead
Cross-origin cookie not keeping people logged in after login	`sameSite: 'lax'` blocks cross-origin fetch, and frontend and backend end up on different hostnames once deployed	Cookie settings now change per environment, `sameSite: 'none'` and `secure: true` when deployed, `'lax'` locally
Fresh database had no seed data, so registration kept failing	The seed scripts had never actually been run against a genuinely empty database before	Confirmed the seed scripts work properly against both a fresh staging database and a patched production database
Two migrations failed the same way on staging and, it turned out, on production too	Migration class names didn't match their required timestamp suffix, and a leftover `name` property on the class kept overriding the fix even after the class name was already corrected	Fixed both migration files properly, and manually applied the corrected SQL straight to production since it had the exact same problem
The frontend image accidentally got deployed into the backend's Container App on staging	A copy-paste mistake in `deploy.yml`'s branch logic set the wrong app name in one spot	Fixed the value, then double-checked with `az containerapp show` that each app was finally running the right image
Staging was costing money even while nobody was using it	`min-replicas: 1` keeps a Container App running the whole time no matter what	Set `min-replicas: 0` on both staging apps, accepting a short cold start on the first request after it's been idle

Uni Textbook Marketplace

In collaboration **with:**

2026 NexusDev - University of Pretoria - COS 301 Capstone Project

This document is version-controlled in the GitHub repository under `/docs/Demo 3/Software Architecture Specifications.pdf`